

APPENDIX A

ALGORITHM PSEUDOCODE

The pseudocodes of proposed Algorithm KP-MLCS.

Algorithm 1 KP-MLCS

Input: Sequences set $T = \{S_1, S_1, \dots, S_d\}$;
Output: $MLCS$ s;
1: **begin**
2: Constructing successor tables ST of T in parallel by Eq. (2);
3: $|MLCS|_{LB} \leftarrow \text{Call Calculating } |MLCS|_{LB} \text{ of } T$
4: $k \leftarrow 0$; $DAG_{KP} \leftarrow \phi$; $D^k \leftarrow \{(0, 0, \dots, 0)\}$;
5: $DAG_{KP}[k] \leftarrow D^k$;
6: **while** $D^k \neq \{(\infty, \infty, \dots, \infty)\}$ **do**
7: Calculating $D^k \rightarrow D^{k+1}$ by querying ST in parallel;
8: Calculating D_{nkp1}^{k+1} by Eq. (5) in parallel;
9: Calculating D_{nkp2}^{k+1} by Alg. Fast-NDSorting in parallel;
10: Deleting $D^{k+1} \leftarrow D^{k+1} - D_{nkp1}^{k+1} - D_{nkp2}^{k+1}$;
11: $DAG_{KP}[k+1] \leftarrow D^{k+1}$;
12: **for each** j from $i+1$ to 1 **do**
13: Cascading deletion $DAG_{KP}[j] \leftarrow DAG_{KP}[j] - D_{nkp3}$;
14: **end for**
15: $k \leftarrow k+1$;
16: **end while**
17: Outputting all paths (all $MLCS$ s) in constructed DAG_{KP} ;
18: **end**
19:
20: **Procedure Calculating** $|MLCS|_{LB}$ **of input sequences** T
21: $k \leftarrow 0$; $DAG_{KP} \leftarrow \phi$; $D^k \leftarrow \{(0, 0, \dots, 0)\}$;
22: **while** $D^{k+1} \neq \{(\infty, \infty, \dots, \infty)\}$ **do**
23: Calculating $D^k \rightarrow D^{k+1}$ by a DOP-based MLCS Algorithm ST in parallel;
24: **for each** $p \in D^{k+1}$ **do**
25: Calculating $Score(p)$ by Eq. (4);
26: **end for**
27: Sorting $p \in D^{k+1}$ and selecting the top 10% points with minimum $Score(p)$ in D^{k+1} to store the set D_{min}^{k+1}
28: $D^{k+1} \leftarrow D_{min}^{k+1}$
29: $k \leftarrow k+1$
30: **end while**
31: **return** $|MLCS|_{LB} \leftarrow k$;

Algorithm 2 ISDominated($node_t, p, dim_index$)

Input: A node of Index-Tree $node_t$, domination point p with d dimension, and dimension index dim_index of p ;
Output: The result of being dominated $flag$;
1: **if** $dim_index == d$ **then**
2: **return** True;
3: **end if**
4: **for each** $e \in node_t$ **do**
5: **if** $e.value < p[dim_index]$ **then**
6: $node_t \leftarrow e.link$;
7: $flag \leftarrow \text{ISDominated}(node_t, p, dim_index + 1)$;
8: **if** $flag$ **then**
9: **return** True;
10: **end if**
11: **else**
12: **break**;
13: **end if**
14: **end for**
15: **return** False;

Algorithm 3 ConsIndexTree($node_t, p, d_index$)

Input: A node of Index-Tree $node_t$, domination point p with d dimension, and dimension index d_index of p ;
Output: The constructed Index-Tree;
1: **if** $d_index == d$ **then**
2: **return** null;
3: **end if**
4: **if** $node_t == \text{null}$ **then**
5: $node_t \leftarrow$ Initializing an empty LinkedList;
6: Initializing an element e ;
7: $e.value \leftarrow p[d_index]$;
8: $e.link \leftarrow \text{ConsIndexTree}(\text{null}, p, d_index + 1)$;
9: Inserting e into $node_t$ at the position 0;
10: **else**
11: $insert_index \leftarrow 0$;
12: **for each** $e \in node_t$ **do**
13: **if** $e.value < p[d_index]$ **then**
14: **continue**;
15: **else**
16: **break**;
17: **end if**
18: $insert_index \leftarrow insert_index + 1$;
19: **end for**
20: **if** $node_t[insert_index].value == p[d_index]$ **then**
21: $e \leftarrow node_t[insert_index]$;
22: $e.link \leftarrow \text{ConsIndexTree}(e.link, p, d_index + 1)$;
23: **else**
24: Initializing an element e' ;
25: $e'.value \leftarrow p[d_index]$;
26: $e'.link \leftarrow \text{ConsIndexTree}(\text{null}, p, d_index + 1)$;
27: Inserting e' into $node_t$ at the index $insert_index$;
28: **end if**
29: **end if**
30: **return** $node_t$;

Algorithm 4 Fast-NDSorting(D^{k+1})

Input: The point set D^{k+1} in $(k+1)$ -th level;
Output: The dominated point set D_{nkp2}^{k+1} ;
1: $D_{nkp2}^{k+1} \leftarrow \emptyset$;
2: // Subsec. 4.3.3 Step 1
3: $P_{S1} \leftarrow$ Select those points in D^{k+1} with the smallest values in any dimensions;
4: $P_{S2} \leftarrow D^{k+1} - P_{S1}$
5: // Subsec. 4.3.3 Step 2
6: $index_tree(a) \leftarrow \text{null}$;
7: **for each** $p \in P_{S1}$ **do**
8: $index_tree(a) \leftarrow \text{ConsIndexTree}(index_tree(a), p, 0)$
9: **end for**
10: **for each** $p \in P_{S2}$ **do**
11: $flag \leftarrow \text{ISDominated}(index_tree(a), p, 0)$
12: **if** $flag$ **then**
13: $D_{nkp2}^{k+1} \leftarrow D_{nkp2}^{k+1} \cup \{p\}$;
14: $P_{S2} \leftarrow P_{S2} - \{p\}$;
15: **end if**
16: **end for**
17: // Subsec. 4.3.3 Step 3
18: $index_tree(b) \leftarrow \text{null}$;
19: **for each** $p \in P_{S2}$ **do**
20: $index_tree(b) \leftarrow \text{ConsIndexTree}(index_tree(b), p, 0)$
21: **end for**
22: **for each** $p \in P_{S2}$ **do**
23: $flag \leftarrow \text{ISDominated}(index_tree(b), p, 0)$
24: **if** $flag$ **then**
25: $D_{nkp2}^{k+1} \leftarrow D_{nkp2}^{k+1} \cup \{p\}$;
26: **end if**
27: **end for**
28: **return** D_{nkp2}^{k+1} ;

APPENDIX B

TIME AND SPACE COMPLEXITIES

For each sequence S_i of T over alphabet Σ with length n , the $O(|\Sigma|n)$ time is needed for constructing its successor table. The main operations in serially constructing the DAG_{KP} consist of the following: 1) computing $D^k \rightarrow D^{k+1}$ ($1 \leq k \leq |MLCS| - 1$). Let the dominants of the given sequences be D . The time complexity of the procedure $D^k \rightarrow D^{k+1}$ is far lower than $O(|D|)$. 2) deleting the non-critical points D_{nkp1}^{k+1} . The time complexity of the operation is $O(|MLCS| \cdot |D^k|_{max})$, where $|D^k|_{max}$ indicates the maximum of $\{D^k | 1 \leq k \leq |MLCS| - 1\}$. 3) deleting the non-critical points D_{nkp2}^{k+1} . Suppose there are n_{k+1} points with d dimensions in D^{k+1} ($0 \leq k \leq |MLCS| - 1$), and let the totalities of the points in index trees (a) and (b) be n_a and n_b , respectively, where $n_a, n_b \ll n_{k+1}$. The time complexity of constructing index trees (a) and (b) is $O(d \cdot n_a \log(n_a) + d \cdot n_b \log(n_b)) \ll O(d \cdot n_{k+1} \log(n_{k+1}))$ by Alg. 3 `RecurConsIndexTree`. The time complexity of Alg. 2 `ISDominated` is less than $O(d \cdot n_a + (d - 1) \cdot n_b) \ll O(d \cdot n_{k+1})$. As $O(d \cdot n_{k+1} \log(n_{k+1})) + O(d \cdot n_{k+1}) = O(d \cdot n_{k+1} \log(n_{k+1}))$, the complexity of deleting the non-critical points D_{nkp2}^{k+1} by Alg. 4 `Fast-NDSorting` is far lower than $O(d \cdot n_{k+1} \log(n_{k+1}))$. 4) the complexity of the cascading deletion in DAG_{KP} is $O(|MLCS|^2 \cdot |D^k|_{max})$.

Therefore, the total time complexity of the serial version of the proposed algorithm $KP\text{-}MLCS$ is $O(d|\Sigma|n) + O(|D|) + O(|MLCS| \cdot |D^k|_{max}) + O(d \cdot n_{k+1} \log(n_{k+1})) + O(|MLCS|^2 \cdot |D^k|_{max}) = O(|D|)$ as $O(d|\Sigma|n)$, $O(d \cdot n_{k+1} \log(n_{k+1}))$ and $O(|MLCS|^2 \cdot |D^k|_{max}) \ll O(|D|)$. The time complexity of our parallel algorithm $KP\text{-}MLCS$ is $\frac{O(|D|)}{N_p} + T_{com}$, where N_p is the number of parallel threads and T_{com} is the communication time cost between threads. So our $KP\text{-}MLCS$ time complexity is linear in $O(|D|)$.

Similarly, the storage space of successor tables is $O(d|\Sigma|n)$, and the storage space of the DAG_{KP} with $|V|$ points and $|E|$ edges is $O(d|V| + |E|)$. The space complexity of $KP\text{-}MLCS$ is $O(d|\Sigma|n + d|V| + |E|) = O(d|V| + |E|)$ as $O(d|\Sigma|n) \ll O(d|V| + |E|)$.

APPENDIX C

SOURCE CODE OF $KP\text{-}MLCS$

The source code, experimental datasets, Appendixes A-D and supplementary materials of the algorithm $KP\text{-}MLCS$ can be found online: <https://github.com/kp-mlcs/KP-MLCS>.

APPENDIX D

PROOFS OF THEOREMS 1-3

D.1 Proof of Theorem 2

Proof: During the level-wise iterative calculation of $D^k \rightarrow D^{k+1}$ for constructing $MLCS\text{-}DAG$ ($0 \leq k \leq |MLCS| - 1$) (see Fig. 2), an immediate predecessor of the point p ($p \in D^{k+1}$) would be any point in D^k , which has $2^{|D^k|} - 1$ possibilities. At the same time, there are two cases: 1) p has only one immediate predecessor in D^k , that is, p is only calculated once, which has a $\frac{|D^k|}{2^{|D^k|} - 1}$ probability; 2) p has two or more immediate predecessors in D^k . In this case, p is calculated at

least twice, which has a $\frac{2^{|D^k|} - 1 - |D^k|}{2^{|D^k|} - 1}$ probability. Therefore, the minimum expected total number of repeated calculations for point p in D^{k+1} can be computed as follows:

$$\frac{|D^k|}{2^{|D^k|} - 1} \times 1 + \frac{2^{|D^k|} - 1 - |D^k|}{2^{|D^k|} - 1} \times 2 = 2 - \frac{|D^k|}{2^{|D^k|} - 1} \quad (6)$$

D.2 Proof of Theorem 3

Proof: We know that there is no dominated point residing in D^k of the $MLCS\text{-}DAG$ ($1 \leq k \leq |MLCS| - 1$) for the dominant-point based $MLCS$ algorithms. For a deleted dominated point p in D^k , it must be compared with $[1, |D^k| - 1]$ points in D^k before it is deleted, that is, compared with $|D^k|/2$ points on average. On the other hand, since p should be an immediate successor or successor to one or more other compared points in D^k , p must reappear in D^{k+w} ($1 \leq w \leq |MLCS| - k$). At this point, p must be compared with $|D^{k+w}|/2$ points on average to determine whether it is retained or deleted again in D^{k+w} . As w is a random value between 1 and $|MLCS| - k$, the probability of p reappearing in $(k + w)$ -level is $1/(|MLCS| - k)$. So, the expectation of M can be formulated as follows:

$$\mathbb{E}(M) = \frac{|D^k|}{2} + \sum_w \frac{1}{|MLCS| - k} \cdot \frac{|D^{k+w}|}{2}, (|D^k|, |D^{k+w}| \in \mathbb{N}) \quad (7)$$

$$\because |D^k| > 1 \text{ and } \sum_w \frac{1}{|MLCS| - k} \cdot \frac{|D^{k+w}|}{2} > 1$$

$$\therefore \mathbb{E}(M) > \frac{|D^k|}{2} + 1 \geq 2 \quad (1 \leq w \leq |MLCS| - k) \quad \blacksquare$$

D.3 Detailed Proof of Theorem 1

Proof: Since the character c appears $|S_i|_c$ times in sequence S_i ($1 \leq i \leq d$), the upper bound of the number N' of the non-repeating points corresponding to the character c in $MLCS\text{-}DAG$ is $\prod_{i=1}^d |S_i|_c$. For all the characters in alphabet Σ , there are at most $\sum_{c \in \Sigma} \prod_{i=1}^d |S_i|_c$ non-repeated points. Therefore, the upper bound N'_{max} is equal to $\sum_{c \in \Sigma} \prod_{i=1}^d |S_i|_c$.

Based on the related literature [14], [15] and our extensive experimental verification, we derive the following properties: 1) $|S_i|$ is proportional to $|MLCS|$ and $|MLCS| \approx |S_i|/2$; 2) $|D^k|$ increases almost exponentially with $|\Sigma|^k$ when $1 \leq k \leq |MLCS|/2$, and it exponentially decreases with the opposite trend when $|MLCS|/2 < k \leq |MLCS| - 1$. In addition, the peak of $|D^k|$ is at around $|MLCS|/2$ -levels. Thus, the upper bound N_{max} is as follows:

$$N_{max} = 2 \cdot (|\Sigma|^1 + |\Sigma|^2 + \dots + |\Sigma|^{\frac{|MLCS|}{2}}) \approx \frac{2 \cdot |\Sigma| \cdot (|\Sigma|^{\frac{|S_i|}{4}} - 1)}{|\Sigma| - 1} \quad (8)$$

Suppose that each $|S_i|_c$ ($1 \leq i \leq d$) has the same probability of $|S_i|/|\Sigma|$. Then the upper bound N'_{max} can be written as

$$N'_{max} = \sum_{c \in \Sigma} \prod_{i=1}^d |S_i|_c \approx |\Sigma| \cdot \left(\frac{|S_i|}{|\Sigma|} \right)^d = \frac{|S_i|^d}{|\Sigma|^{d-1}} \quad (9)$$

Based on Eqs. (8) and (9), when $|\Sigma| > 1$ with given d , N_{max} is an exponential function of $|S_i|$, while N'_{max} is a power function of $|S_i|$. Therefore, we can infer the fact that when $|S_i|$ is greater than or equal to a certain threshold, $N_{max} \gg N'_{max}$. For example, when $|\Sigma|=4$, $d=3$, and $|S_i|=40$, $N_{max}=2796200$ and $N'_{max}=4000$, i.e., $N_{max} \gg N'_{max}$. $\blacksquare$